

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
(МОСКОВСКИЙ ПОЛИТЕХ)

КУРСОВОЙ ПРОЕКТ

по курсу
«Разработка веб-приложений»

ТЕМА

«Разработка веб-приложения для управления музыкальными
релизами независимого артиста с личным кабинетом и
аналитикой прослушиваний»

Выполнил: Осыкин Кирилл Андреевич

Группа: 241-3211

Проверил: Кружалов Алексей Сергеевич

Москва, 2026

width=!,height=!,pages=-

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	4
ГЛАВА 1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ	5
1.1 Обзор существующих программных продуктов для управления музыкальными релизами	5
1.2 Анализ программных инструментов разработки веб-приложений	6
1.3 Формулировка цели и задач работы	8
ГЛАВА 2. ПРОЕКТИРОВАНИЕ ВЕБ-ПРИЛОЖЕНИЯ	10
2.1 Анализ целевой аудитории	10
2.2 Описание функциональности приложения	10
2.3 Проектирование модели данных	10
2.4 Разработка макетов страниц	12
ГЛАВА 3. РАЗРАБОТКА ВЕБ-ПРИЛОЖЕНИЯ	13
3.1 Разработка базовой структуры приложения и вёрстка	13
3.2 Реализация модуля авторизации	13
3.3 Интеграция картографических сервисов	13
ЗАКЛЮЧЕНИЕ	15

ВВЕДЕНИЕ

Актуальность темы. В условиях активного развития цифровой музыкальной индустрии независимые артисты получают всё больше возможностей для самостоятельной публикации и продвижения своих музыкальных произведений. Современные стриминговые площадки и музыкальные дистрибьюторы позволяют размещать треки без участия крупных лейблов, однако вместе с этим у исполнителя возникает необходимость систематизировать информацию о релизах, треках, площадках публикации, обложках, аудиофайлах и статистике прослушиваний. Использование разрозненных таблиц, заметок и файловых хранилищ затрудняет управление музыкальным каталогом и анализ результатов публикации. В связи с этим разработка веб-приложения с личным кабинетом независимого артиста, медиатекой и аналитикой прослушиваний является актуальной практической задачей.

Степень разработанности проблемы. На рынке существуют различные сервисы для музыкальной дистрибуции и анализа статистики, такие как DistroKid, TuneCore, CD Baby, ONErpm, а также личные кабинеты артистов в рамках отдельных музыкальных платформ. Подобные решения предоставляют инструменты для публикации релизов, мониторинга прослушиваний и управления музыкальным контентом. Однако большинство таких сервисов ориентировано на коммерческую дистрибуцию, работу с реальными стриминговыми площадками и монетизацию. Для учебного проекта представляет интерес создание упрощённой веб-системы, которая имитирует функциональность личного кабинета артиста и позволяет реализовать основные механизмы веб-разработки: аутентификацию, авторизацию, CRUD-интерфейсы, загрузку файлов и агрегирование данных.

Объект исследования — процессы управления музыкальными релизами независимого артиста в цифровой среде.

Предмет исследования — программные средства и методы разработки веб-приложения для хранения, обработки и анализа информации о музыкальных релизах, треках, площадках публикации и статистике прослушиваний.

Целью данной работы является проектирование и реализация клиент-серверного веб-приложения для управления музыкальными релизами независимого артиста с личным кабинетом, медиатекой, системой ролей пользователей и аналитикой прослушиваний.

ГЛАВА 1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

1.1 Обзор существующих программных продуктов для управления музыкальными релизами

В настоящее время независимые артисты активно используют цифровые сервисы для публикации музыкальных произведений на стриминговых площадках, ведения каталога релизов и анализа статистики прослушиваний. К таким сервисам относятся музыкальные дистрибьюторы, агрегаторы и аналитические платформы, предоставляющие личные кабинеты для исполнителей.

Основными функциями подобных систем являются загрузка музыкальных файлов, добавление информации о релизах, управление обложками, указание даты публикации, выбор музыкальных площадок, а также просмотр статистики по прослушиваниям. Проведем сравнение нескольких существующих решений в таблице 1.1.

Таблица 1.1 – Сравнительный анализ сервисов для музыкальных релизов

Название	Ключевая особенность	Модель доступа
DistroKid	Быстрая публикация музыкальных релизов на стриминговых площадках	Платная подписка
TuneCore	Дистрибуция музыки, сбор доходов и предоставление инструментов для артистов	Платная подписка или комиссия
CD Baby	Публикация релизов и помощь независимым артистам в монетизации музыки	Платное размещение
ONErpm	Дистрибуция, продвижение и аналитика музыкальных релизов	Комиссионная модель
Яндекс Музыка для артистов	Просмотр статистики исполнителя внутри экосистемы Яндекс Музыка	Бесплатный доступ для артистов

Анализ показал, что существующие сервисы в основном ориентированы на реальную дистрибуцию музыкального контента и работу с коммерческими площадками. Такие платформы обладают большим количеством функций, однако часть из них может быть избыточной для начинающего независимого артиста, которому требуется простое приложение для систематизации собственных релизов, треков, ссылок на площадки и статистики прослушиваний.

Таким образом, актуальной является разработка учебного веб-приложения, имитирующего функциональность личного кабинета независимого артиста. Такое приложение позволит управлять музыкальными релизами, хранить ин-

формацию о треках, загружать обложки и аудиофайлы, указывать площадки публикации и анализировать основные показатели прослушиваний.

1.2 Анализ программных инструментов разработки веб-приложений

Для реализации веб-приложения необходимо выбрать инструменты для разработки серверной части, пользовательского интерфейса, базы данных и механизма хранения файлов. Так как приложение предполагает наличие личного кабинета, авторизации, ролей пользователей, CRUD-интерфейсов, загрузки файлов и аналитики, целесообразно использовать классическую клиент-серверную архитектуру.

В качестве основного языка программирования для разработки проекта выбран *Python*. Данный язык обладает понятным синтаксисом, большим количеством библиотек и активно применяется при создании веб-приложений, сервисов обработки данных и административных панелей.

Для разработки серверной части выбран микрофреймворк *Flask*. Он позволяет создавать веб-приложения с маршрутизацией запросов, обработкой форм, подключением шаблонов, работой с базой данных и организацией пользовательских сессий. В отличие от более крупных фреймворков, Flask предоставляет разработчику гибкость в выборе архитектуры приложения и хорошо подходит для учебного курсового проекта.

В качестве возможных инструментов разработки были рассмотрены следующие технологии:

- **Язык программирования:** Python. Используется для реализации серверной логики приложения, обработки пользовательских запросов, работы с базой данных и формирования аналитических показателей.
- **Backend-фреймворк:** Flask. Применяется для создания маршрутов, обработки HTTP-запросов, реализации личного кабинета пользователя, аутентификации, авторизации и CRUD-операций.
- **Шаблонизатор:** Jinja2. Используется для генерации HTML-страниц на стороне сервера и передачи данных из Python-кода в интерфейс приложения.

- **Frontend-инструменты:** HTML, CSS, JavaScript и Bootstrap. Данные технологии позволяют разработать страницы личного кабинета, формы добавления релизов, таблицы треков, карточки музыкальных релизов и блоки аналитики.
- **Работа с базой данных:** SQLAlchemy. Данная библиотека позволяет описывать модели данных на языке Python и выполнять операции с базой данных через ORM-подход.
- **База данных:** SQLite или PostgreSQL. SQLite может использоваться на этапе разработки и тестирования, так как не требует отдельной настройки сервера базы данных. PostgreSQL подходит для дальнейшего развития проекта и развёртывания приложения.
- **Хранение файлов:** локальное файловое хранилище. В рамках курсового проекта достаточно реализовать загрузку обложек релизов и аудиофайлов или демо-фрагментов треков в директорию приложения.

Для реализации пользовательского интерфейса целесообразно использовать HTML, CSS и Bootstrap. Такой подход позволяет быстро создать адаптивные страницы без разработки сложного отдельного frontend-приложения. Шаблонизатор Jinja2 дает возможность выводить данные из базы данных непосредственно в HTML-шаблоны: например, список релизов артиста, карточки треков, статистику прослушиваний и ссылки на музыкальные площадки.

Для хранения данных приложения могут использоваться SQLite или PostgreSQL. В рамках курсового проекта SQLite является удобным вариантом, так как база данных хранится в одном файле и не требует отдельной установки серверной СУБД. При этом структура приложения может быть спроектирована таким образом, чтобы в дальнейшем при необходимости перейти на PostgreSQL. Основными сущностями системы являются пользователь, артист, релиз, трек, площадка публикации и статистика прослушиваний.

Для реализации авторизации и разграничения прав доступа может использоваться механизм пользовательских сессий Flask. Роли пользователей позволяют ограничивать доступ к определённым страницам и операциям: администратор может управлять всеми данными системы, артист — своими релизами и треками, а обычный пользователь или гость — просматривать доступную информацию.

Таким образом, использование Python и Flask позволяет реализовать все обязательные функции проекта: аутентификацию и авторизацию пользователей, разграничение прав доступа по ролям, CRUD-интерфейс, загрузку файлов, хранение данных и агрегирование статистики прослушиваний.

1.3 Формулировка цели и задач работы

На основе проведенного анализа предметной области и существующих программных продуктов была сформулирована цель курсового проекта.

Цель работы — разработка веб-приложения для управления музыкальными релизами независимого артиста с личным кабинетом, медиатекой и аналитикой прослушиваний.

Разрабатываемое приложение должно предоставлять пользователю возможность создавать и редактировать музыкальные релизы, добавлять треки, загружать обложки и аудиофайлы, указывать музыкальные площадки публикации, а также просматривать агрегированную статистику прослушиваний.

Для достижения поставленной цели необходимо решить следующие **задачи**:

1. Провести обзор существующих сервисов для музыкальной дистрибуции, управления релизами и анализа статистики прослушиваний;
2. Проанализировать программные инструменты, подходящие для разработки веб-приложения на языке Python с использованием фреймворка Flask;
3. Спроектировать архитектуру веб-приложения и структуру базы данных для хранения пользователей, релизов, треков, площадок публикации и статистики;
4. Разработать интерфейс личного кабинета независимого артиста с использованием HTML, CSS, Bootstrap и шаблонов Jinja2;
5. Реализовать механизмы аутентификации, авторизации и разграничения прав доступа пользователей;
6. Реализовать CRUD-интерфейс для управления музыкальными релизами, треками и площадками публикации;

7. Реализовать загрузку и хранение файлов, включая обложки релизов и аудиофайлы или демо-фрагменты треков;
8. Реализовать модуль аналитики, включающий подсчет общего количества прослушиваний, определение популярных треков и сравнение статистики по музыкальным площадкам;
9. Провести тестирование разработанного веб-приложения и подготовить отчет о проделанной работе.

ГЛАВА 2. ПРОЕКТИРОВАНИЕ ВЕБ-ПРИЛОЖЕНИЯ

2.1 Анализ целевой аудитории

Основными пользователями разрабатываемой системы являются сотрудники логистических компаний. Выделяются две ключевые роли:

- **Диспетчер-логист** — сотрудник, ответственный за распределение заказов и контроль их выполнения в реальном времени.
- **Курьер** — конечный исполнитель, использующий мобильную версию приложения для навигации по маршруту.

Для каждой роли проведено профилирование по методу персонажей (Personas) для минимизации когнитивной нагрузки на интерфейс.

2.2 Описание функциональности приложения

Функциональные возможности системы определяются через взаимодействие пользователя с системой. Основной бизнес-кейс представлен в виде *User Story*: «Я, как диспетчер, хочу импортировать список адресов, чтобы система автоматически построила кратчайший маршрут для 5 курьеров».

На рисунке 2.1 представлена диаграмма вариантов использования (Use Case).

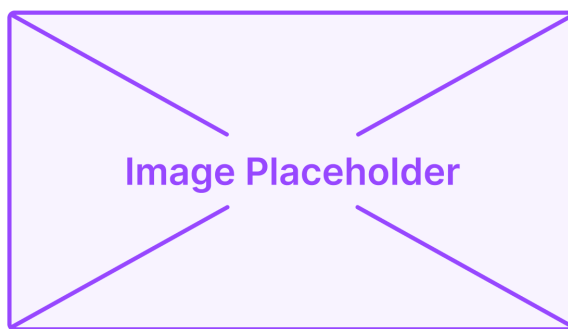


Рисунок 2.1 – Диаграмма вариантов использования системы

2.3 Проектирование модели данных

При выборе архитектуры базы данных был проведен сравнительный анализ эффективности различных решений, опирающийся на данные о произ-

водительности современных СУБД [1].

Структура таблицы заказов представлена в табл. 2.1.

Таблица 2.1 – Структура таблицы заказов (orders)

Поле	Тип	Ограничение	Описание
id	UUID	PRIMARY KEY	Уникальный идентификатор заказа
customer_id	UUID	FOREIGN KEY	Ссылка на таблицу клиентов
address	TEXT	NOT NULL	Полный адрес доставки
lat	NUMERIC	NOT NULL	Широта точки (WGS-84)
lng	NUMERIC	NOT NULL	Долгота точки (WGS-84)
weight	FLOAT	CHECK > 0	Вес груза, кг
volume	FLOAT	NOT NULL	Объем груза, м ³
time_from	TIME	NOT NULL	Начало окна доставки
time_to	TIME	NOT NULL	Конец окна доставки
priority	INT	DEFAULT 1	Приоритет (1–5)
status	VARCHAR	'pending'	Статус (ожидание, в пути, выполнен)
courier_id	UUID	NULL	Назначенный исполнитель
created_at	TIMESTAMP	DEFAULT NOW()	Время создания записи
updated_at	TIMESTAMP	DEFAULT NOW()	Время последнего изменения
is_fragile	BOOLEAN	FALSE	Хрупкий груз
contact_phone	VARCHAR	NOT NULL	Телефон получателя
price	MONEY	NOT NULL	Стоимость доставки
external_id	VARCHAR	UNIQUE	ID из внешней системы (1C/ERP)

Продолжение таблицы 2.1

Поле	Тип	Ограничение	Описание
notes	TEXT	NULL	Комментарий для курьера
attempts	INT	DEFAULT 0	Количество попыток доставки

Общая стоимость маршрута для службы доставки рассчитывается следующим образом:

$$C_{total} = \sum_{i=1}^n (L_i \cdot P_{fuel} + T_i \cdot W_{hour}) + S_{fixed}, \quad (1)$$

где

L_i — протяженность i -го участка маршрута, км;

P_{fuel} — стоимость топлива на 1 км пути;

T_i — время в пути на участке;

W_{hour} — почасовая ставка оплаты труда курьера;

S_{fixed} — постоянные издержки на обслуживание автопарка.

2.4 Разработка макетов страниц

Макеты страниц

ГЛАВА 3. РАЗРАБОТКА ВЕБ-ПРИЛОЖЕНИЯ

3.1 Разработка базовой структуры приложения и вёрстка

В соответствии с задачей 3.1, была реализована компонентная структура фронтенд-части на базе библиотеки **React.js**. Использование хуков (*Hooks*) позволило эффективно управлять состоянием формы заказа. Пример базового компонента представлен в листинге 1.

Листинг 1 – Пример структуры компонента карточки заказа

```
1 // Комментарий
2 const OrderCard = ({ order }) => {
3   return (
4     <div className="order-card">
5       <h3Заказ> #{order.id}</h3>
6       <pАдрес>: {order.address}</p>
7       <button onClick={() => addToRoute(order.id)}>
8         Добавить в маршрут
9       </button>
10    </div>
11  );
12 };
```

3.2 Реализация модуля авторизации

Согласно задаче 3.2, в системе реализована ролевая модель доступа. Для обеспечения безопасности используется протокол **JWT** (JSON Web Token). При входе в систему сервер генерирует токен, содержащий поле *role*, которое может принимать значения *'dispatcher'* или *'courier'*.

3.3 Интеграция картографических сервисов

Ключевым этапом разработки (задача 3.4) стала интеграция API **OpenStreetMap** через библиотеку *Leaflet*. Для решения транспортной задачи был выбран жадный алгоритм с последующей оптимизацией методом локального поиска.

Данные о маршруте передаются в формате JSON (см. листинг 2).

Как видно из листинга 2, каждая точка содержит порядковый номер (*sequence*), что позволяет курьеру следовать по оптимальному пути, рассчитанному на сервере.

Листинг 2 – Структура данных оптимизированного маршрута

```
1 {  
2   "route_id": "r-101",  
3   "courier_id": "u-45",  
4   "waypoints": [  
5     {"lat": 55.75, "lng": 37.61, "sequence": 1},  
6     {"lat": 55.80, "lng": 37.70, "sequence": 2}  
7   ],  
8   "total_distance_km": 12.4  
9 }
```

ЗАКЛЮЧЕНИЕ

В результате проведенного исследования подтверждена гипотеза о том, что автоматизация логистических процессов на базе специализированного веб-приложения позволяет существенно повысить эффективность работы служб экспресс-доставки. Теоретический анализ показал, что классическая задача маршрутизации транспортных средств (VRP) в современных условиях требует адаптации под динамические изменения городского трафика и специфику «последней мили».

На основе полученных при разработке данных можно сформулировать следующие **выводы**:

1. Использование открытых ГИС-технологий и компонентного подхода (React/Node.js) обеспечивает необходимую архитектурную гибкость, позволяя интегрировать алгоритмы оптимизации без зависимости от проприетарных платных сервисов.
2. Проектирование модели данных с поддержкой пространственных расширений (PostGIS) является критическим условием для масштабируемости системы, так как позволяет эффективно обрабатывать геозапросы в режиме реального времени.
3. Внедрение эвристических методов построения маршрутов демонстрирует устойчивый экономический эффект, выраженный в снижении совокупного пробега автотранспорта и сокращении времени планирования рейсов диспетчером.

Рекомендации и предложения. Для практического применения разработанного решения в деятельности малых логистических компаний предлагается использовать модель облачного развертывания. Это обеспечит бесперебойный доступ к интерфейсу управления и мобильному клиенту курьера при минимальных затратах на ИТ-инфраструктуру.

Перспективы углубления темы. В дальнейшем исследовании планируется развитие функциональности системы за счет внедрения методов предиктивной аналитики для прогнозирования времени прибытия (ETA) с учетом исторических данных о заторах, а также разработки модуля динамиче-

ской корректировки маршрута при поступлении срочных заявок в процессе выполнения рейса.

При разработке пояснительной записки к курсовому проекту строго соблюдались требования [2], определяющие структуру и правила оформления научно-исследовательских работ.

СПИСОК ЛИТЕРАТУРЫ

- [1] Петров, П. П. Анализ производительности современных СУБД [Текст] / П. П. Петров, С. С. Сидоров // Вестник информационных технологий. — 2024. — № 2. — С. 15–22.
- [2] ГОСТ 7.32-2017. Отчет о научно-исследовательской работе. Структура и правила оформления [Текст]. — 2017.